- In the same method as centroids initialization, but with a different MapReduce job;
- In the same MapReduce job we described in the previous section, with the reducer that becomes in charge of also performing classification at the same time it chooses which centroids should be kept. In particular, the reducer would get from mappers all the list of points assigned to each centroid, so rather than cycling through all dataset points again, it could re-use these lists (we'll see later in this section how it will need to combine them).

The first option is rather easier, but naïve; implementing the classification step at least in the same method as the choice of centroids gives us two main advantages:

- We can, in theory, reuse the same mappers, which already hold on to their shards of data, by just passing them the list of canopy centroids;
- When we perform classification, there is an issue with the centroids filtering we perform in the reducers that, so far, we have been able to ignore: running centroid filtering and classification together allows us to solve this issue efficiently.

Figure 13.10 shows what might happen when we filter out one or more centroids during the reduce step in our algorithm above; we haven't mentioned this issue until now because it only affects the algorithm when we assign points to canopies, while it is irrelevant when we only care about centroids, like in the k-means initialization steps.